

Département de génie informatique et génie logiciel

Cours INF1900:
Projet initial de système embarqué

Travaux pratiques 7 et 8

Production de librairies statiques et stratégies de débogage

Par l'équipe

No 2728

Noms:

Amélie Bazinet
Ayhem Cheikh
Vincent Dufresne
Félix Tétreault

Date:
16 juillet 2026

Partie 1 : Description de la librairie

can

La librairie can permet d'utiliser le convertisseur analogique/numérique qu'on relie au microcontrôleur. Elle permet de lire une tension analogique sur une entrée du microcontrôleur, qu'elle convertit en une valeur numérique qui peut être utilisée par le programme. Elle est constituée de deux fichiers : can.h, qui contient la déclaration de la classe, et can.cpp, qui contient l'implémentation des méthodes. can.cpp contient l'implémentation du constructeur, du destructeur et de la méthode lecture. Le constructeur de la classe can ne prend aucun paramètre et initialise le module ADC. Pour utiliser la librairie, le constructeur doit être initialisé avant la méthode lecture.

La méthode lecture permet d'effectuer une conversion sur le canal analogique sélectionné. Cette méthode lit une tension analogique sur une entrée du microcontrôleur et convertit cette tension en valeur numérique. Le signal à mesurer doit être relié à une entrée analogique du microcontrôleur. Le paramètre pos de la méthode représente l'entrée analogique du microcontrôleur. Ce paramètre doit être compris entre 0 et 7, puisqu'il correspond aux entrées analogiques ADC0 à ADC7. Une fois la conversion terminée, la valeur numérique est retournée dans les registres ADCL et ADCH sous la forme d'un uint16_t. La méthode est bloquante, car elle attend que le bit ADIF indique la fin de la conversion pour continuer son exécution. Avec un prescaler de 64, cette opération dure environ 832 cycles d'horloge, soit 104 µs.

Communication

La librairie Communication contient les fonctionnalités de communication UART du microcontrôleur pour la transmission de données entre un périphérique et le microcontrôleur, ce qui permet entre autres de faciliter le débogage du robot. La librairie est constituée de deux fichiers : Communication.hpp, qui contient la déclaration de la classe et ses méthodes, et Communication.cpp, qui contient l'implémentation des méthodes.

La méthode initialisationUART configure le module USART0 pour permettre la communication UART. La méthode doit être appelée avant toute transmission pour configurer correctement le module. Les broches PD0 et PD1 sont réservées pour la communication UART et ne peuvent pas être utilisées sur la carte mère. initialisationUART configure les registres :

- UBBR0H et UBBR0L pour configurer le baud rate à 2400 bauds;
- UCSR0B (RXEN0, TXEN0, UCSZ02) pour activer la réception et la transmission;
- UCSR0C pour transmettre par 8 bits, sans parité et avec un bit de stop.

La méthode transmissionUART envoie un octet via UART. Elle prend en paramètre une donnée uint8_t à transmettre. La fonction est bloquante, car elle compte une boucle d'attente while qui attend que le registre soit libre avant d'écrire la donnée sur le registre. transmissionUART utilise les registres :

- UCSR0A pour vérifier si le registre est libre (UDRE0 = 0);
- UDR0. L'écriture de la donnée dans ce registre déclenche la transmission de l'octet (donnée).

La méthode transmissionChaine transmet une chaîne de caractères via UART, octet

par octet, en faisant appel à transmissionUART. Finalement, la méthode transmissionFormatee transmet une chaîne formatée comme printf. Elle facilite l'envoi de messages du microcontrôleur au périphérique (ordinateur) en convertissant les valeurs en texte avant de les transmettre et simplifie donc l'affichage de messages de débogage. La fonction utilise un tampon de 64 caractères maximum par message.

debug

La librairie intègre un mécanisme de débogage conditionnel à l'aide du macro DEBUG_PRINT(comm, format, ...). Celui-ci permet l'affichage de messages de diagnostic sans modifier le comportement du programme final lorsque le mode débogage est désactivé. L'usage d'un macro conditionnel constitue un choix pertinent, car il sépare le code de développement du code de production. Cette approche évite d'avoir à supprimer manuellement les instructions de débogage avant la compilation finale du programme. Ce mécanisme est contrôlé par l'option de compilation -DDEBUG, activée par le debug du Makefile.

Lorsque DEBUG est défini, DEBUG_PRINT appelle la méthode transmissionFormatee(format, ...) de la classe Communication passé en argument. Il devient alors possible d'émettre des messages formatés semblables à ceux d'un printf. Cela facilite l'observation de l'état du programme pendant le développement, notamment pour afficher des valeurs de variables, confirmer le passage dans certaines sections du code ou suivre le déroulement d'une séquence d'exécution.

Lorsque DEBUG n'est pas défini, le macro est remplacé par une instruction vide « do {} while (0) ». Dans ce cas, aucun code de transmission n'est généré à la compilation et il n'y a donc aucun coût d'exécution, aucune attente active, et aucune occupation de mémoire liée à l'affichage de messages de débogage.

En mode actif, le comportement de DEBUG_PRINT dépend directement de la fonction transmissionFormatee, dont il hérite les caractéristiques. Le macro est donc bloquant, puisque la transmission UART utilisée pour afficher le message attend que le registre de transmission soit prêt avant chaque envoi. Son temps d'exécution est variable et dépend de la longueur du message formaté ainsi que de la vitesse de communication UART. En mode inactif, il est non bloquant et son temps d'exécution est nul, puisqu'aucune instruction utile n'est produite.

Le mécanisme DEBUG_PRINT ne nécessite aucune ISR et n'affecte directement aucun registre du microcontrôleur. En revanche, lorsqu'il est actif, les registres utilisés indirectement sont ceux manipulés par la librairie Communication, notamment les registres du module UART. DEBUG_PRINT dépend donc de l'initialisation préalable de la communication série par initialisationUART(). Si cette initialisation n'a pas été effectuée avant son utilisation en mode DEBUG, les messages ne pourront pas être transmis correctement.

Del

La librairie Del facilite le contrôle d'une DEL sur la carte mère. L'utilisation de cette classe ne nécessite aucune interruption ISR. Elle est constituée de deux fichiers : Del.hpp, qui contient la déclaration de la classe, et Del.cpp, qui contient l'implémentation des méthodes.

La DEL peut être branchée à n'importe quelle broche de la carte mère. Le constructeur de la classe Del prend en paramètre une référence vers n'importe quel port PORTx ainsi que les numéros des deux broches utilisées. Lors de la construction de l'objet,

ces broches sont configurées en sortie à l'aide du registre de direction DDRx.

Dans le fichier Del.cpp, les valeurs des broches sont comprises entre 0 et 7. Si une autre valeur est fournie, seuls les 3 bits de poids faibles sont conservés.

La méthode allumerCouleur permet de modifier la couleur de la DEL en activant les broches appropriées selon la couleur choisie (éteint, vert ou rouge). Les couleurs sont contenues dans une classe Couleur. Cette méthode est non bloquante et n'utilise aucun délai. La méthode allumerCouleurAmbre permet de simuler la couleur ambre en alternant rapidement entre le rouge et le vert en boucle. Cette méthode a un temps d'exécution de 10 millisecondes, puisqu'elle utilise la fonction _delay_ms() avec un délai de 5 millisecondes entre chaque changement de couleur.

InterruptionExterne

La librairie InterruptionExterne facilite la configuration, la gestion et l'utilisation des interruptions externes INT0, INT1 et INT2 du microcontrôleur ATmega, associées aux broches PD2, PD3 et PB2 respectivement. Elle permet de manipuler les interruptions sans avoir à manipuler directement les registres matériels dans le code. Elle est divisée en deux fichiers : InterruptionExterne.hpp, qui contient la déclaration de la classe et de ses méthodes, et InterruptionExterne.cpp, qui contient l'implémentation des fonctions.

La fonction initialisation() configure une interruption externe. Elle prend en paramètre indiceInt, l'indice de l'interruption (0, 1 ou 2), ainsi que detectionInterruption, le mode de déclenchement : 0 pour un déclenchement au niveau bas, 1 pour tout changement logique, 2 au front descendant et 3 au front montant. La fonction initialisation agit sur les registres suivants :

- EIMSK : activation l'interruption sélectionnée (INT0, INT1 ou INT2).
- EICRA : configuration du mode de détection à l'aide des bits ISCx0 et ISCx1.
- DDRB ou DDRD : configuration la broche associée à l'interruption en entrée.

Avant d'appliquer la nouvelle configuration, les bits ISCx0 et ISCx1 sont remis à zéro afin d'éviter qu'une configuration précédente influence le nouveau mode de détection. La fonction commence par désactiver les interruptions globales à l'aide de cli() afin d'éviter qu'une interruption survienne pendant la configuration des registres. Elle se termine par la réactivation de l'interruption et des interruptions globales. Cette fonction est non bloquante et s'exécute en temps constant, puisqu'elle consiste uniquement en des opérations sur des registres. Lorsque la librairie est utilisée en mode interruption, le programme appelant doit déclarer le vecteur d'interruption correspondant, soit ISR(INT0_vect), ISR(INT1_vect) ou ISR(INT2_vect) selon l'interruption configurée. La librairie configure uniquement le matériel. Le traitement de l'événement doit être défini dans l'ISR du programme principal.

desactiverInterruptionsGlobales() et activerInterruptionsGlobales() utilisent cli() et sei() respectivement pour désactiver ou activer les interruptions globales. desactiverInterruption() et activerInterruption() permettent de désactiver ou d'activer une seule interruption sans perturber les autres interruptions déjà actives.

changerDetectionInterruption permet de modifier le mode de déclenchement d'une interruption déjà configurée, en changeant dynamiquement detectionInterruption.

La librairie permet aussi un mode scrutation (polling) sans ISR grâce à interruptionEnAttente() et effacerFlag(). Ces deux fonctions sont non bloquantes et se

complètent.

- interruptionEnAttente() lit EIFR pour détecter un événement directement dans la boucle principale.
- effacerFlag() doit être appelée après le traitement ou avant de réactiver les interruptions globales pour éviter des déclenchements parasites.

memoire_24

La classe Memoire24CXXX de la librairie memoire_24 fournit une interface pour l'écriture et la lecture séquentielle de données sur une mémoire EEPROM externe du microcontrôleur via le protocole I^2C. Cette classe ne nécessite aucune ISR et utilise les broches PC1 et PC0. La classe est constituée de deux fichiers, memoire_24.hpp qui contient la déclaration de la classe et mémoire_24.cpp qui contient l'implémentation des méthodes de la classe. Toutes les opérations faites par la classe sont bloquantes, car elles ont des boucles d'attente active lors de la transmission ou de l'écriture.

La classe offre deux versions de la méthode lecture, pour lire un octet à la fois ou pour en lire plusieurs de façon séquentielle. Avant de débuter la lecture, la méthode vérifie que la mémoire n'est plus occupée par un cycle d'écriture. La lecture se fait ensuite via le protocole I^2C et se termine lorsque la condition d'arrêt est atteinte. Finalement, le domaine des paramètres de la fonction est restreint. Pour la méthode de lecture à plusieurs octets, la longueur donnée doit correspondre à un espace valide en mémoire.

La classe contient aussi deux versions de la méthode écriture pour écrire un octet à la fois ou pour en écrire plusieurs de façon séquentielle. Avant de commencer une opération d'écriture, la méthode communique avec la mémoire pour valider qu'aucun cycle d'écriture est en cours. L'adresse mémoire à modifier est transmise sur 16 bits, suivie des informations à écrire. Pour l'écriture à plusieurs octets, les données sont segmentées selon la taille d'une page mémoire afin d'éviter le repliement des données. Comme pour la lecture, la fonction attend la fin d'un cycle d'écriture avant de commencer. Une condition d'arrêt marque la fin de la transmission. Une condition d'arrêt met aussi fin à la transmission des données. Le domaine des paramètres d'entrée est le même que celui de la méthode lecture.

Minuterie

La librairie Minuterie permet d'utiliser les minuteries 0, 1 et 2 du microcontrôleur. Elle permet de gérer des délais et d'utiliser les comparateurs pour déclencher des interruptions et produire des signaux PWM. Les méthodes de cette librairie ne sont pas bloquantes. Elle est constituée de deux fichiers : Minuterie.hpp, qui contient la déclaration des classes, méthodes et énumérations, et Minuterie.cpp, ainsi que l'implémentation des méthodes.

La librairie contient trois classes (Minuterie0, Minuterie1 et Minuterie2) et contient des énumérations qui permettent de configurer plus facilement les registres des minuteries. Ces énumérations servent à définir le fonctionnement de la minuterie, le prescaler, le mode de sortie des comparateurs ainsi que le comparateur à utiliser.

La méthode configurer est présente dans chacune des trois classes de Minuterie. Elle permet de configurer les registres de contrôle de la minuterie selon le mode de fonctionnement, le prescaler et le mode de sortie des comparateurs choisis. Il est important d'appeler cette méthode avant l'utilisation de la minuterie afin d'initialiser les différents registres. Cette méthode met à jour prescaler_ avec la valeur choisie.

Pour une minuterie X (0,1 ou 2), la méthode configurer() configure les registres :

- TCCRxA : configuration du mode de fonctionnement de la minuterie et du comportement des sorties de comparaison.
- TCCRxB : sélection du prescaler (et configuration du mode pour minuterie 1)
- TCCRxC : initialisation à 0 (seulement utilisé pour la minuterie 1).

Les méthodes activerComparateurA et activerComparateurB permettent de convertir un temps (ms) en une valeur de comparaison écrite dans les registres du comparateur. Elles utilisent la fréquence du microcontrôleur ainsi que la valeur du prescaler pour calculer le nombre de cycles nécessaires. Les méthodes activerInterruptionComparateurA, desactiverInterruptionComparateurA, activerInterruptionComparateurB et desactiverInterruptionComparateurB permettent d'activer ou de désactiver les interruptions associées aux comparateurs. Il est important que le prescaler ait été configuré avant leur utilisation. Ces méthodes permettent de déclencher les routines d'interruption (ISR).

La méthode partirPWM est présente dans les classes Minuterie0 et Minuterie2. Elle initialise les registres nécessaires afin de produire un signal PWM sur les sorties de comparaison de la minuterie. Cette méthode prend en paramètre deux ratios compris entre 0 et 100. Pour Minuterie2, il est important de noter que la méthode partirPWM désactive d'abord les interruptions globales à l'aide de cli(), puis les réactive avec sei() après la configuration. Le prescaler est mis a 8 lors de la configuration de la minuterie dans la fonction.

La classe Minuterie1 contient une méthode différente. La méthode partirMinuterie permet de démarrer une minuterie sur une plus longue durée, puisque la minuterie 1 peut compter jusqu'à 65536 cycles, ce qui correspond à un maximum de 8388 ms par intervalle. Pour ce faire, cette méthode met le prescaler à 1024 dans tous les cas. Si le temps demandé est supérieur à cette valeur, la méthode divise la durée totale en plusieurs itérations complètes et conserve le reste dans un variable. Cette méthode désactive aussi temporairement les différentes ISR.

Finalement, la méthode miseAJourCompteur permet de faire le suivi de la minuterie longue durée. Elle doit être appelée dans la routine d'interruption, par exemple dans ISR(TIMER1_COMPA_vect). À chaque interruption, la méthode vérifie s'il reste encore des itérations complètes ou un temps restant à exécuter. Lorsque la durée est écoulée, elle met la variable estMinuterieExpiree à true.

PontH

La librarie PontH utilise la librairie Minuterie pour contrôler les moteurs à l'aide de signaux PWM générés par la minuterie Minuterie2. Elle gère la direction et le ratio du signal PWM pour chaque roue. Le ratio, qui représente le pourcentage de la vitesse totale à laquelle la roue doit rouler, doit avoir une valeur comprise entre 0 et 100, puisque c'est un pourcentage. Les méthodes ne sont pas bloquantes et ne nécessitent aucune ISR. Les broches utilisées sont :

- PD4 : direction de la roue gauche (0 : avance, 1 : recule).
- PD5 : direction de la roue droite (0 : avance, 1 : recule).
- PD6 : activation de la roue gauche (d'une vitesse définie par le ratio choisi).
- PD7 : activation de la roue droite (d'une vitesse définie par le ratio choisi).

La méthode initialiserRegistre() doit être appelée avant de configurer le PWM des roues et configure les ports PD4 à PD7 en sortie.

Plusieurs méthodes sont ensuite définies pour contrôler le mouvement du robot. La méthode configurer(directionGauche, ratioGauche, directionDroite, ratioDroite) est la plus versatile et permet de définir le mouvement du robot en lui donnant la direction des roues souhaitée et le ratio de la vitesse de chacune des roues. Cette méthode permet alors de configurer n'importe quel mouvement souhaité, peu importe le sens ou la vitesse. La méthode configurer est par la suite utilisée pour définir le comportement des méthodes avancer, reculer, tournerDroite, tournerGauche et arrêter :

- avancer(ratio) et reculer(ratio) prennent seulement en argument le ratio de vitesse des roues et définissent une même direction pour les deux roues (0 pour avancer, 1 pour reculer).
- tournerDroite(ratio et tournerGauche fait tourner le robot sur lui-même en faisant rouler les deux roues à la même vitesse (selon le ratio donné), mais en faisant tourner une roue vers l'avant et une roue vers l'arrière (selon la méthode).
- arreter() met le ratio des deux roues à 0, ce qui freine les roues et arrête le mouvement.

Partie 2 : Modifications apportées au Makefile

Makefile de la librairie

Ce Makefile permet de générer une librairie statique (libstatique.a) à partir des fichiers .cpp de la librairie. Plusieurs modifications ont été faites dans le Makefile de départ pour le rendre fonctionnel et plus simple. Tout d'abord, plusieurs sections inutiles ont été retirées. Les règles liées à la création de fichiers .hex et .elf ou à l'installation sur le microcontrôleur (install dans .PHONY) ont été supprimées, puisque ce Makefile sert uniquement à produire une librairie statique (.a). Le projet source (PRJSRC) a été changé pour prendre comme source tous les fichiers .cpp du dossier de la librairie à l'aide de $(wildcard *.cpp).

Une règle pour produire un fichier libstatique.a à partir des fichiers .o a été créée à l'aide de l'outil avr-ar, qui regroupe les fichiers objets dans un seul fichier .a. Certaines variables ont aussi été retirées car elles n'étaient pas nécessaires (OBJCOPY, AVRDUDE, HEXTRG etc.). Finalement, les règles concernant la compilation de fichiers en C ont été retirées, puisque notre librairie compte uniquement des fichiers .cpp. Si on souhaitait ajouter des librairies .c, il faudrait rajouter ces règles.

Makefile du code

Le Makefile du code permet de compiler seulement l'exécutable (main.cpp, bien qu'ici ce soit un code bidon) qui utilise la librairie statique générée par le premier Makefile. Plusieurs modifications ont été faites dans le Makefile de départ. Tout d'abord, le Makefile ne crée pas de librairie et lie plutôt l'exécutable (main.cpp compilé en main.o) avec le fichier libstatique.a généré par le Makefile. La librairie est incluse via -I$(LIBDIR) où LIBDIR=../lib. Et est liée à main.o ainsi :

```
$(TRG): $(MAINOBJ) $(LIBS)
        $(CC) $(LDFLAGS) -o $@ $^ -lm          $^ inclut main.o et libstatique.a
```

Un support pour debug a aussi été ajouté, permettant au code d'avoir des sections conditionnelles pour le débogage (décrit plus haut dans debug). Finalement, tout comme le Makefile de la librairie, les règles concernant les fichiers en C ont été retirées par souci de simplicité.